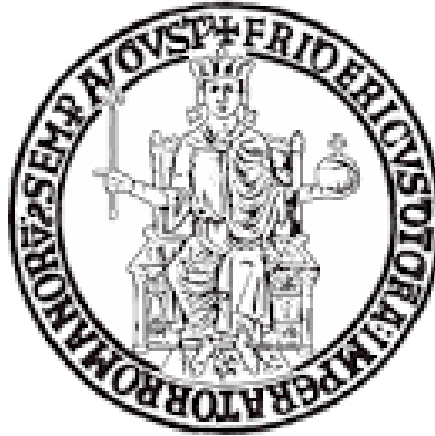




UNIVERSITA' DEGLI STUDI DI
NAPOLI FEDERICO II

Progetto Neural Network and Deep Learning

Neapolitan Cards Recognition

Anno accademico 2025/2026

Corso informatica

Autori:

Mirko Prevenzano DE5000044

Davide Di Pierro DE5000001

Fabrizio Quaranta DE5000029

Indice

1	Introduzione	2
1.1	Tecnologie utilizzate	2
2	Progettazione dataset	2
2.1	Strategia di Acquisizione delle Immagini	3
2.1.1	Utilizzo di Più Mazzi	3
2.1.2	Condizioni di Acquisizione Controllate	3
2.1.3	Strategia di Split del Dataset	3
2.1.4	Anonimizzazione dei Dati	4
2.2	Data Augmentation	4
2.3	Riepilogo delle Scelte Metodologiche	5
3	Progettazione e Implementazione del Modello	7
3.1	Preprocessing e normalizzazione dei dati	7
3.2	Caricamento dei dati	7
3.3	Definizione del modello	8
3.3.1	Scelta dell'architettura	8
3.4	Scelta funzione d'errore	9
3.5	Fase di addestramento iterativo	10
3.6	Differenze di implementazione dei due modelli	10
4	Prima sperimentazione	11
4.0.1	Configurazione degli Iperparametri	11
4.1	Modello a 40 classi	12
4.1.1	Analisi delle metriche	13
4.2	Modello a 14 classi	13
4.2.1	Report Numeri	14
4.2.2	Report Lettere	15
4.2.3	Analisi sulle metriche	15
5	Sperimentazione finale	17
5.0.1	Configurazione degli Iperparametri e modifiche strutturali	17
5.0.2	Configurazione degli Iperparametri	17
5.1	Modello a 40 classi (ResNet18)	19
5.2	Modello a 14 classi (ResNet18)	20
5.3	Modello a 40 classi (ResNet50)	24
5.4	Modello a 14 classi (ResNet50)	25
6	Explainable AI	28
6.1	Analisi dei risultati	29
6.1.1	Guida come analizzare i risultati	29
6.1.2	Risultati	29
6.2	Il modello generalizza?	30
7	Conclusioni	31
7.1	Approccio a 40 classi VS Approccio a 14 classi	31
7.2	ResNet-18 vs ResNet-50	32

1 Introduzione

Il progetto *Neapolitan Cards Recognition* nasce dall'esigenza di colmare un vuoto osservato nel panorama della Computer Vision. Analizzando le piattaforme del settore (come Roboflow), abbiamo notato che c'è una netta predominanza delle carte da gioco internazionali, a fronte di una quasi totale assenza di carte regionali (tra cui le napoletane). Mossi dalla voglia di coprire questo buco, è nato Neapolitan Cards Recognition.

L'obiettivo principale è dunque la creazione di un sistema di riconoscimento robusto e di un dataset pubblico che possa servire da riferimento per la cultura ludica locale. Il progetto è concepito per essere versatile e integrabile in diversi scenari applicativi, tra cui:

- *Robotica* Implementazione del modello in agenti robotici in grado di identificare le carte in modo che possano interagire fisicamente durante una partita.
- *Arbitraggio Automatizzato* Monitoraggio di tavoli da gioco reali tramite visione artificiale per dirigere le partite tra due giocatori reali.

1.1 Tecnologie utilizzate

- *Kaggle* Utilizzata come piattaforma principale per l'addestramento dei modelli. La scelta è stata adottata dato che Kaggle offre 30 ore di GPU al mese, fornendo due GPU NVIDIA Tesla T4. Ciò ha permesso di addestrare le varie versioni del modello in tempi estremamente ridotti.
- *Roboflow* Piattaforma fondamentale che ha permesso di gestire l'intero dataset, annotarlo e offrirlo alla community, rendendo il dataset annotato pubblico.
- *Captum* Libreria dedicata alla *Explainable AI* per PyTorch. Integrata per analizzare il processo decisionale del modello e assicurare l'assenza di Data Leakage.
- *Python e PyTorch* Python è stato scelto per le sue numerose librerie per il calcolo scientifico. PyTorch è il framework di machine learning utilizzato per definire e addestrare il modello.

2 Progettazione dataset

Nella progettazione del dataset è stato affrontato il rischio del data leakage. Ogni operazione svolta è il risultato di decisioni prese analizzando il seguente paper *Don't push the button! Exploring data leakage risks in*.

Il data leakage si verifica quando informazioni che non dovrebbero essere disponibili durante l'addestramento influenzano il modello, portando a valutazioni artificialmente ottimistiche che non si mantengono su dati realmente nuovi.

Nel contesto del riconoscimento di carte napoletane e considerando il paper indicato, il data leakage può manifestarsi in molteplici forme:

- Condizioni di acquisizione sistematicamente diverse per classi diverse.
- Metadati dei file che rivelano informazioni sulle classi
- Split inadeguati che condividono informazioni tra il training e il test.
- Preprocessing applicato prima della separazione dei dati.
- Feature spurie correlate con le etichette.

2.1 Strategia di Acquisizione delle Immagini

L'acquisizione delle immagini costituisce la fase più critica per la prevenzione della *data collecting leakage*.

2.1.1 Utilizzo di Più Mazzi

Per garantire la capacità di generalizzazione del modello, sono stati utilizzati più mazzi di carte napoletane fisicamente distinti. Questa scelta risponde a diverse esigenze:

1. *Variabilità delle caratteristiche fisiche* Ogni mazzo presenta differenze nella qualità di stampa, tonalità dei colori, livello di usura e texture della carta.
2. *Prevenzione dell'overfitting su caratteristiche specifiche* Se tutte le carte provenissero dallo stesso mazzo, il modello potrebbe imparare pattern specifici di quel mazzo (es. imperfezioni) invece delle caratteristiche generali delle carte.

2.1.2 Condizioni di Acquisizione Controllate

Le fotografie sono state scattate utilizzando un *treppiede fotografico* per garantire:

- *Stabilità* Eliminazione del rumore dovuto al movimento della fotocamera.
- *Ripetibilità* Angolazione e distanza costanti per tutte le carte dello stesso mazzo.
- *Qualità uniforme*

Tuttavia, per prevenire il *leakage dovuto al design dell'acquisizione*, sono state introdotte variazioni controllate tra i diversi mazzi:

- V1.** *Ambientazione diversa per ogni mazzo* Ogni mazzo è stato fotografato in un ambiente differente (es. tavolo in legno, superficie bianca, sfondo neutro), introducendo variabilità negli sfondi.
- V2.** *Illuminazione variabile* Le condizioni di luce sono state variate tra i mazzi per evitare che il modello associ specifiche condizioni di illuminazione a particolari carte.
- V3.** *Randomizzazione dell'ordine di acquisizione* Le carte non sono state fotografate in sequenza per seme o valore, ma in ordine casuale, per prevenire correlazioni temporali.

2.1.3 Strategia di Split del Dataset

La suddivisione del dataset in training, validation e test è stata progettata per prevenire ogni forma di *split-related leakage*.

Lo split è stato effettuato immediatamente dopo l'acquisizione delle immagini e prima dell'augmentation. In particolare, ogni mazzo di carte è assegnato a un singolo split: train, validation o test.

Questo per garantire che le caratteristiche di ogni mazzo non sono condivise tra loro, oltre che permettere di testare il modello su carte mai viste prima.

Sono stati utilizzati mazzi completi da 40 carte ciascuno, così da evitare classi sbilanciate.

2.1.4 Anonimizzazione dei Dati

L'anonimizzazione serve a prevenire il *label leakage*, eliminando qualsiasi informazione che possa rivelare direttamente o indirettamente la classe di appartenenza.

Nel progetto non si verifica label leakage grazie a tre fattori:

1. *Etichette derivate dalla struttura delle cartelle* Il caricamento dei dati avviene tramite `torchvision.datasets.ImageFolder`, che assegna le etichette in base al nome della sottocartella (es. `train/1 A/`, `train/10 D/`), non in base al nome del file immagine.
2. *Il modello riceve solo pixel* il `DataLoader` fornisce al modello esclusivamente coppie (`tensor_image`, `etichetta`). Nessun nome di file, percorso o metadato viene passato alla rete neurale durante il training o la valutazione.
3. *Metadati EXIF assenti*: le immagini non contengono metadati EXIF (data, ora, fotocamera, GPS). Inoltre, la libreria `torchvision` durante il caricamento legge esclusivamente i valori dei pixel, ignorando qualsiasi metadato residuo.

Anche nei casi in cui i nomi dei file non sono stati rinominati (es. nel test set), ciò non costituisce un rischio poiché `ImageFolder` non espone i nomi dei file al modello: il metodo `__getitem__` restituisce solo il tensore dell'immagine e l'indice numerico della classe.

2.2 Data Augmentation

La data augmentation (rotazioni, flip, variazioni di luminosità/contrasto, crop casuali, ...) è una tecnica per aumentare la dimensione effettiva del dataset e migliorare la generalizzazione.

Data la dimensione limitata del dataset originale, l'applicazione della data augmentation è estesa anche al validation set per ottenere una stima delle prestazioni più stabile e statisticamente significativa. Un set di validazione troppo piccolo porta a metriche molto sensibili alla singola immagine.

Nel train set la data augmentation è più robusta, con diverse varianti elencate di seguito:

- *Flip Vertical* Ribalta l'immagine lungo l'asse orizzontale. È fondamentale simulare carte viste dal lato opposto del tavolo;
- *Rotation* ($\pm 90^\circ$ e $\pm 15^\circ$) Le rotazioni a 90° simulano carte disposte lateralmente, mentre le rotazioni lievi ($\pm 15^\circ$) simulano l'imprecisione naturale nel posizionamento manuale delle carte;
- *Greyscale* Converte l'immagine in scala di grigi, rendendo il modello meno dipendente dalle dominanti cromatiche e più attento a forme e contorni;
- *Saturation & Brightness* Varia l'intensità dei colori e la luminosità. Simulano diverse condizioni di illuminazione ambientale;
- *Noise* Introduce una grana casuale sui pixel per simulare il rumore elettronico dei sensori delle fotocamere in condizioni di bassa luminosità;

- *Blur* Applica una sfocatura controllata per simulare errori di messa a fuoco o micro-movimenti durante lo scatto, migliorando la capacità del modello di generalizzare su immagini non perfettamente nitide;

Mentre il set di addestramento beneficia di tutte le varianti per massimizzare la capacità di astrazione, per il validation set si è scelto di limitare l'augmentation alle sole componenti geometriche. Questa scelta permette di seguire il dominio specifico delle carte da gioco, nel quale l'orientamento non deve alterare l'identità semantica della singola carta. Le trasformazioni geometriche applicate non sono interpretate come distorsioni artificiali, ma come rappresentazioni reali in cui le carte possono essere disposte sul tavolo da gioco. Ciò permette quindi al modello di apprendere l'invarianza rotazionale che impone la carta da gioco.

Tutte le versioni aumentate di un'immagine devono rimanere nello stesso split dell'immagine originale.

Attenzione

Se si applica augmentation *prima* dello split, versioni aumentate della stessa immagine possono finire in split diversi, introducendo *split-related leakage*: il modello riconosce porzioni comuni tra training e test, gonfiando artificialmente le prestazioni.

2.3 Riepilogo delle Scelte Metodologiche

Aspetto	Scelta Adottata
Mazzi utilizzati	Più mazzi fisici distinti per introdurre variabilità reale.
Condizioni di acquisizione	Treppiede per stabilità; ambiente e illuminazione variabili tra mazzi; ordine di acquisizione randomizzato.
Anonimizzazione	File rinominati con ID neutri; metadati EXIF rimossi con <code>exiftool</code> .
Strategia di split	Ogni mazzo assegnato interamente a un singolo split; split effettuato prima di preprocessing e augmentation.
Preprocessing	Parametri (media, std) calcolati solo su training set; stessa trasformazione applicata a validation e test.
Data augmentation	Applicata al training set e in parte al validation set, dopo lo split.

Tabella 1: Riepilogo delle scelte metodologiche per la prevenzione del data leakage.

Buona pratica

Gli script e la documentazione completa sono disponibili nel repository del progetto, garantendo la riproducibilità di tutte le operazioni.

Schema di Etichettatura (Codifica Classi)

Valore (Numero)		Seme (Lettera)	
1	Asso	A	Denari
2–7	Valore numerale	B	Coppe
8	Fante	C	Spade
9	Cavallo	D	Bastoni
10	Re		

3 Progettazione e Implementazione del Modello

3.1 Preprocessing e normalizzazione dei dati

Le immagini sono rese tutte della stessa dimensione (225*300).

Normalizzare i dati è un punto fondamentale per garantire che tutte le caratteristiche abbiano la stessa importanza. Per esempio, se una feature ha valori che variano da 0 a 1000, mentre un'altra feature ha valori che variano da 0 a 1, la prima feature potrebbe dominare il modello e influenzare negativamente le prestazioni. Normalizzare i dati aiuta a evitare questo problema e a migliorare la convergenza del modello durante l'addestramento. Inoltre, con i valori normalizzati, il processo di addestramento può essere più veloce e più stabile, poiché i pesi del modello non devono adattarsi a scale diverse per ogni caratteristica (pesi enormi possono causare problemi di overflow o underflow durante l'addestramento).

La tecnica scelta per normalizzare i dati delle immagini è quella più comune: la standardizzazione. Quest'approccio trasforma i dati in modo che abbiano una media di 0 e una deviazione standard di 1.

Per il calcolo della media e della deviazione standard è stato scelto come campione il train set dopo aver effettuato la data augmentation e poi con tali valori (μ e σ) è stata eseguita la trasformazione lineare su tutti gli split del dataset. Nello specifico, per ogni pixel x , il valore normalizzato $\hat{x}$ è stato calcolato come:

$$\hat{x} = \frac{x - \mu}{\sigma}$$

È fondamentale sottolineare che la media e la deviazione standard sono state calcolate esclusivamente sul set di addestramento. Questo approccio serve a evitare il data leakage: il modello non deve avere alcuna informazione preventiva sulla distribuzione statistica dei dati di validazione o di test. Pertanto, questi ultimi sono stati normalizzati utilizzando i parametri derivati dal training set, garantendo che il modello operi su una distribuzione coerente con quella appresa durante la fase di ottimizzazione.

Definito l'approccio per la normalizzazione dei dati, si è passati al preprocessing di questi ultimi.

- *Conversione in tensori* I dati sono stati convertiti in tensori a virgola mobile per rendere l'uso della GPU più efficiente.
- *Applicazione della normalizzazione*

3.2 Caricamento dei dati

Una volta definiti i set di dati e le trasformazioni, il passo successivo riguarda l'implementazione della logica di caricamento.

I dati sono stati organizzati secondo una struttura a cartelle, dove ogni sottocartella rappresenta una classe (es. "1A": Asso di Denari). Questo permette l'utilizzo del dataset *ImageFolder*, che mappa automaticamente il nome della cartella all'etichetta (label) numerica corrispondente.

Per gestire il flusso di dati verso il modello, sono stati utilizzati i *DataLoader*. Questi componenti svolgono tre funzioni fondamentali.

- *Batching* Suddividono il dataset in piccoli gruppi (Batch) di dimensioni fisse. Questo è necessario poiché la memoria della GPU non consentirebbe di caricare l'intero dataset contemporaneamente.
- *Shuffling* Le immagini del training set vengono rimescolate a ogni epoca. Questo impedisce al modello di imparare l'ordine sequenziale delle immagini, forzandolo a concentrarsi esclusivamente sulle caratteristiche visive.
- *Parallelismo* Con l'uso di `num_workers`, il caricamento delle immagini avviene in parallelo sulla CPU mentre la GPU elabora il batch precedente, ottimizzando i tempi di calcolo.

3.3 Definizione del modello

Nella definizione del modello è stato optato l'uso dell'approccio Fine Tuning. Il Fine Tuning è una tecnica di transfer learning che consiste nell'addestrare un modello pre-addestrato su un nuovo dataset specifico per un compito particolare. In pratica, si prende un modello che è stato addestrato su un grande dataset (nel caso in questione, ImageNet per la visione artificiale) e si continua ad addestrarlo su un nuovo dataset più piccolo e specifico per il compito che si vuole risolvere. La scelta di usare il Fine Tuning è una diretta conseguenza del dataset limitato, poiché sfrutta le conoscenze apprese dal modello pre-addestrato su un grande dataset, migliorando così le prestazioni sul nuovo compito specifico.

Nella fase di definizione del modello sono configurati anche gli iperparametri che nelle varie sperimentazioni differiscono. Si spiegheranno i dettagli nel capitolo dedicato alle sperimentazioni.

3.3.1 Scelta dell'architettura

La rete scelta come base per il nostro modello appartiene alla famiglia delle ResNet (Residual Networks). Questa architettura è stata introdotta per risolvere il fenomeno della "degradazione del gradiente" (vanishing gradient), un problema che tende ad aggravarsi all'aumentare del numero di strati (layer) della rete, rendendo difficile l'addestramento di modelli molto profondi.

A differenza delle reti neurali tradizionali, in cui ogni strato tenta di apprendere una mappatura diretta dell'input, la ResNet introduce il concetto di apprendimento residuo. Attraverso le cosiddette skip connections (o connessioni scorciatoia), il segnale in ingresso può saltare uno o più strati; in questo modo, ogni blocco della rete deve apprendere solo il "residuo" (ovvero la differenza tra l'input originale e l'output desiderato). Se uno strato non apporta miglioramenti significativi, la rete può semplicemente trasmettere l'input inalterato, mantenendo stabili le prestazioni.

Nelle fasi di sperimentazione, abbiamo alternato l'uso di due varianti, entrambe pre-addestrate sul dataset ImageNet, adattandole tramite Fine Tuning al dominio specifico delle carte da gioco:

- *ResNet-18* Composta da 18 layer, utilizza blocchi formati da due strati di convoluzione. Grazie alla sua struttura leggera e al numero contenuto di parametri (circa 11 milioni), offre un'elevata velocità di esecuzione e richiede una potenza computazionale ridotta, rendendola ideale per un eventuale impiego su dispositivi mobili.

- *ResNet-50* Composta da 50 layer, questa variante è progettata per gestire una complessità maggiore senza un aumento eccessivo del carico computazionale. Introduce infatti il blocco *Bottleneck* (collo di bottiglia), che sfrutta tre convoluzioni: la prima riduce la dimensionalità dei dati, la seconda elabora le caratteristiche e la terza ripristina la dimensione originale. Questa architettura permette di identificare dettagli visivi più complessi e profondi, offrendo una precisione superiore a scapito di una velocità d'inferenza leggermente inferiore.

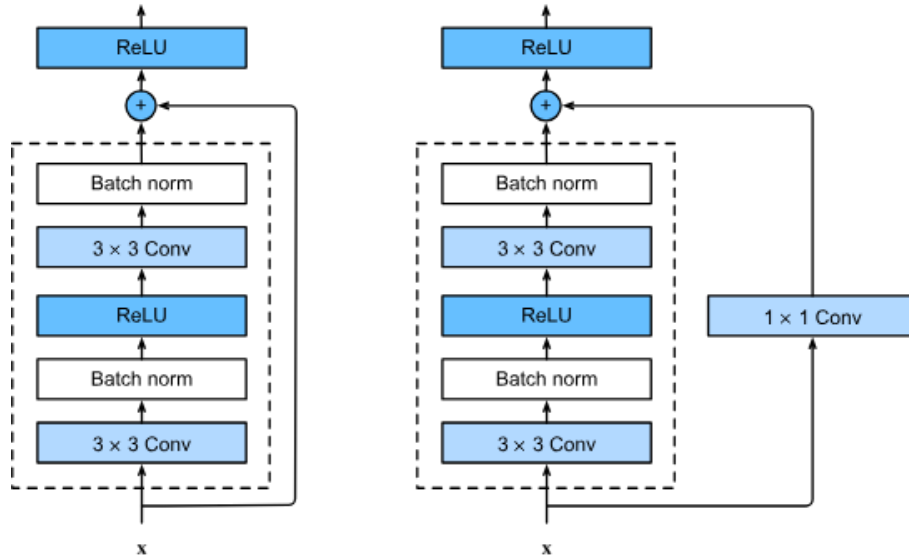


Figura 1: Architettura ResNet e dettaglio del blocco residuo.

3.4 Scelta funzione d'errore

Per l'ottimizzazione del modello, la scelta è ricaduta sulla Cross-Entropy Loss, la funzione di costo standard per i problemi di classificazione multi-classe. Questa metrica misura la divergenza tra la distribuzione di probabilità predetta dal modello e la distribuzione reale delle etichette (rappresentata tramite one-hot encoding), penalizzando in modo logaritmico le previsioni errate, specialmente quando il modello mostra un'elevata confidenza in una classe non corretta.

Affinché l'output della rete sia interpretabile in termini probabilistici, ovvero con valori compresi nell'intervallo $[0, 1]$ e somma totale pari a 1, è necessario normalizzare i punteggi grezzi (logits) prodotti dall'ultimo strato lineare. Poiché i neuroni di output possono assumere qualsiasi valore reale, si introduce la funzione Softmax, definita come segue:

$$\sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

Questa funzione garantisce che le proprietà probabilistiche siano rispettate e che l'argomento del logaritmo all'interno della Cross-Entropy sia sempre positivo. La combinazione di Softmax e Cross-Entropy rappresenta una delle soluzioni più efficienti nel deep learning per ragioni matematiche legate alla stabilità del calcolo dei gradienti. Operando la derivata della funzione di perdita rispetto ai pesi, si ottiene un gradiente estremamente semplificato e "pulito".

$$\frac{\partial L}{\partial z_i} = \hat{y}_i - y_i$$

Dove $\hat{y}_i$ è la probabilità predetta e y_i è l'etichetta reale.

Questo risultato (analogo a quello che si ottiene con la somma dei quadrati nei problemi di regressione) indica che l'errore propagato all'indietro è proporzionale alla sola differenza lineare tra la previsione e il target. Tale semplificazione elimina il rischio di saturazione del gradiente nelle fasi iniziali dell'addestramento, garantendo una convergenza più rapida e stabile dell'algoritmo di ottimizzazione.

3.5 Fase di addestramento iterativo

L'addestramento del modello è stato strutturato in un ciclo iterativo suddiviso in due fasi distinte per ogni epoca.

- *Fase d'addestramento* Si attiva il calcolo del gradiente e l'aggiornamento dei pesi tramite l'ottimizzatore Adam. Inoltre, possono essere abilitati comportamenti specifici (es: dropout).
- *Fase di validation* Il calcolo dei gradienti è disattivato per evitare modifiche ai pesi. Permette di valutare la capacità di generalizzazione del modello su dati non usati durante l'addestramento.

Al termine di ogni epoca, vengono calcolate e memorizzate metriche (accuracy e loss) che permettono di mostrare una history sui risultati del modello dopo ogni epoca.

3.6 Differenze di implementazione dei due modelli

Le differenze tra i due modelli di classificazione riguardano soprattutto la struttura dell'uscita e, di conseguenza, il modo in cui vengono calcolate loss e metriche.

In entrambi i casi la rete usa una ResNet pre-addestrata come già anticipato. La variazione principale avviene nello strato finale 'fully connected' (classificatore), cioè quello che trasforma le feature in punteggi di classe.

Nel modello *single-head* a 40 classi, il classificatore finale viene sostituito con un unico strato lineare con 40 unità in uscita. Ogni unità corrisponde a una delle 40 classi del problema e la rete viene ottimizzata con un'unica CrossEntropyLoss calcolata direttamente sulle 40 classi.

Nel modello *multi-head*, invece, il classificatore finale della ResNet viene rimosso e rimpiazzato da due strati lineari distinti: *head_num* e *head_char*. Entrambe le teste ricevono in input le stesse feature estratte dal backbone, ma producono due insiemi di logits separati: uno per la classificazione del numero (10 classi) e uno per la classificazione del seme/lettera (4 classi). In fase di training si calcolano due loss separate (una per ciascuna testa) che vengono poi combinate (tipicamente sommate) per ottenere la loss totale su cui viene eseguita la backpropagation.

Di conseguenza, anche la valutazione cambia: nel single-head si riportano metriche e matrice di confusione direttamente sulle 40 classi; nel multi-head si riportano metriche separate per numero e seme/lettera (come se fossero due task distinti) e si costruisce anche una valutazione a 40 classi combinando le predizioni delle due teste.

4 Prima sperimentazione

Nella prima sperimentazione sono stati utilizzati 5 mazzi di carte differenti con un totale di 40 foto per mazzo, una per ogni carta. Lo split è avvenuto nel seguente modo:

- Train: 3 mazzi
- Valid: 1 mazzi
- Test: 1 mazzi

Con questo split abbiamo una distribuzione di tipo [60% 20% 20%].

Dopo aver effettuato il processo di data augmentation di seguito è mostrata la nuova distribuzione:

- Train: 1320 immagini
- Valid: 240 immagini
- Test: 40 immagini

4.0.1 Configurazione degli Iperparametri

Il processo di addestramento è stato configurato con una serie di iperparametri scelti per bilanciare la velocità di convergenza e la stabilità del modello.

Iperparametro	Descrizione e Motivazione
Batch Size	<i>32</i> – Definisce quante immagini vengono caricate in memoria contemporaneamente (approccio <i>mini-batch</i>). Ottimizza l'uso della GPU e stabilizza il gradiente.
Num Classes	<i>40</i> – Rappresenta il numero totale di categorie (10 valori per 4 semi).
Num Classes number	<i>10</i> – Classi relative al valore della carta (1–10).
Num Classes char	<i>4</i> – Classi relative al seme della carta (A–D).
Epochs	<i>30</i> – Numero massimo di iterazioni complete sul dataset.
Learning Rate	<i>0.001</i> – Regola l'entità degli aggiornamenti dei pesi.

Tabella 2: Sintesi degli iperparametri di addestramento.

La rete usata in questo primo tentativo è ResNet50

4.1 Modello a 40 classi

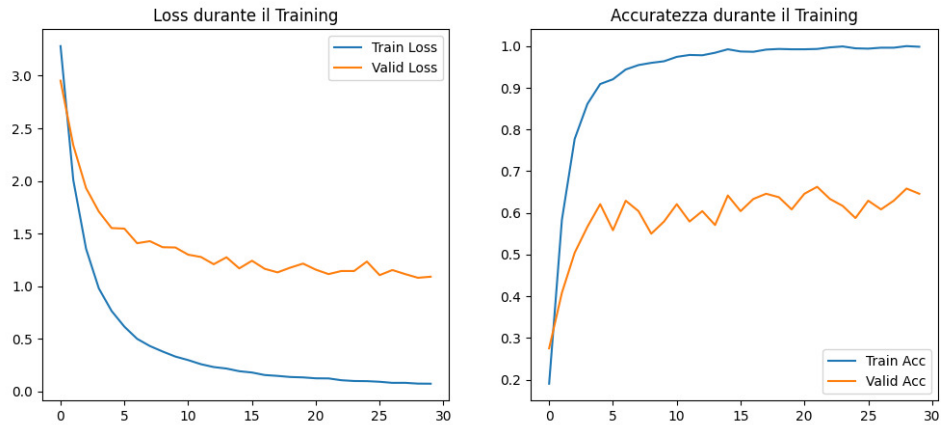


Figura 2: Andamento della Loss - Modello 40 classi (Tentativo 1)

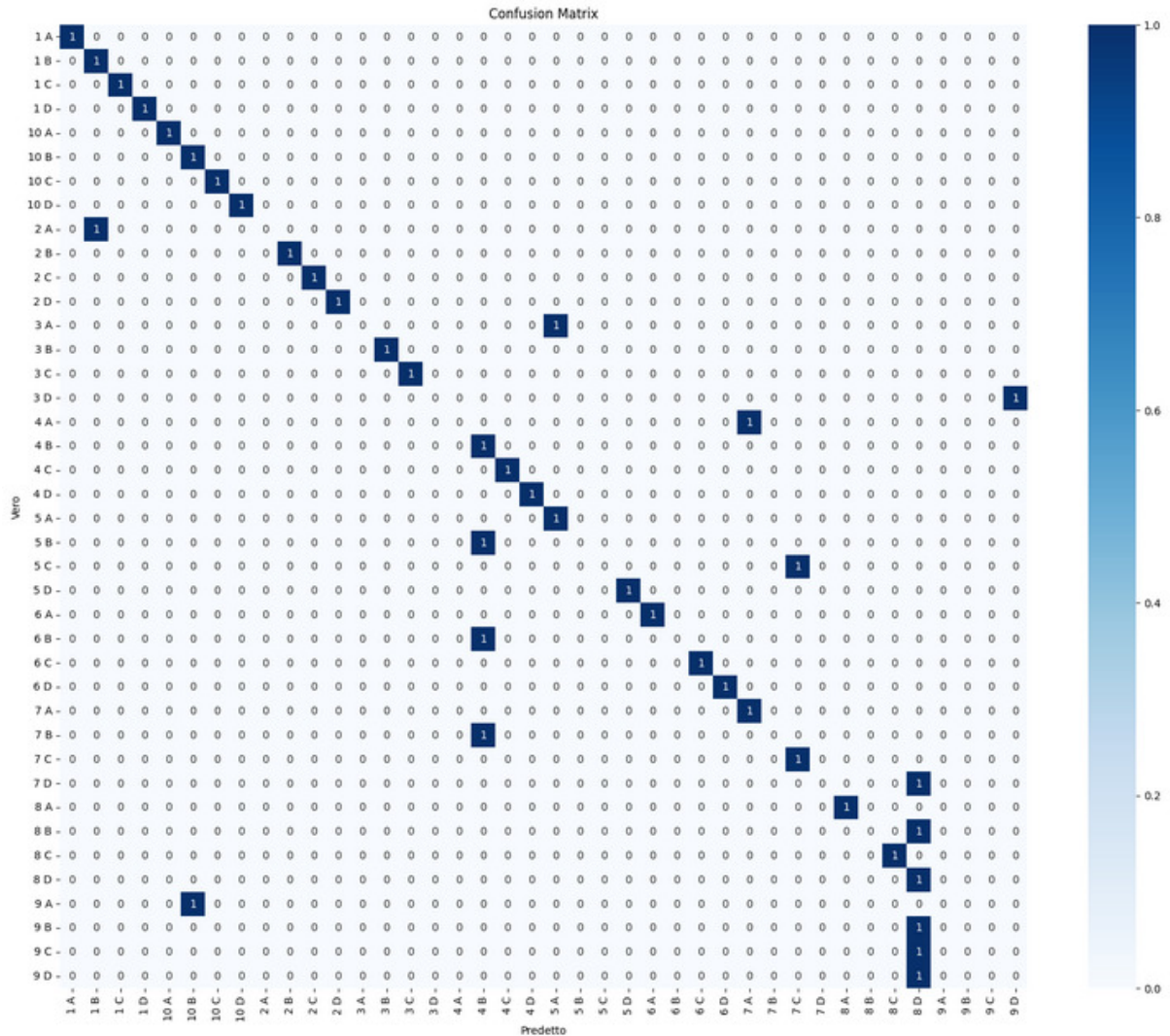


Figura 3: Matrice di Confusione - Modello 40 classi (Tentativo 1)

Report su Test Set La Tabella 3 riassume le metriche globali ottenute sul Test Set per il modello a 40 classi.

Metrica	Valore	Interpretazione
Accuracy	<i>0.65</i>	Il modello classifica correttamente il 65% delle immagini totali.
Macro Avg Precision	<i>0.55</i>	Media della precisione calcolata indipendentemente per ogni classe.
Macro Avg Recall	<i>0.65</i>	Capacità media del modello di individuare le istanze positive.
Macro Avg F1-Score	<i>0.58</i>	Bilanciamento tra precisione e recall, indicativo della robustezza globale.

Tabella 3: Metriche globali del test set (Modello 40 Classi).

4.1.1 Analisi delle metriche

Dall'analisi delle metriche, in particolare dalla funzione loss, si osserva una forte divergenza tra le prestazioni sul set di addestramento e quelle sul set di validazione. Mentre l'accuratezza di training raggiunge valori prossimi al 100%, l'accuratezza di validazione si stabilizza intorno al 65%. Questo comportamento è indicato con il termine overfitting: il modello tende a memorizzare i dettagli specifici del training set piuttosto che generalizzare le caratteristiche semantiche delle carte da gioco.

Nonostante l'applicazione di tecniche di regolarizzazione e data augmentation, la limitata dimensione del dataset originale e l'elevata capacità della rete (ResNet50) hanno favorito una convergenza molto rapida sui dati noti, limitando la capacità di astrazione sul validation set. Dalla matrice di confusione si osserva che il modello commette diversi errori di classificazione. Inoltre, il numero di immagini del test set è troppo basso non fornisce informazioni statistiche.

4.2 Modello a 14 classi

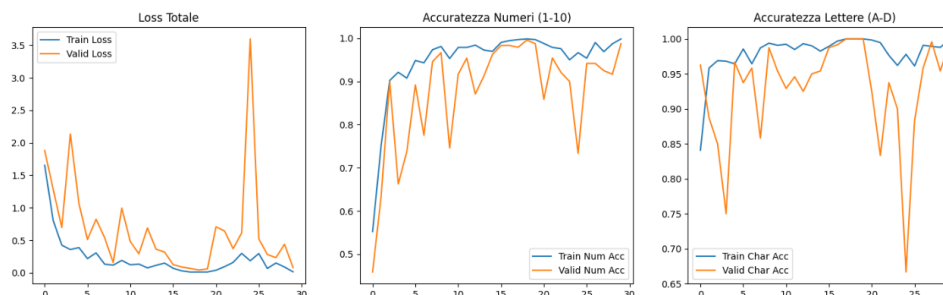


Figura 4: Andamento della Loss - Modello 14 classi

4.2.1 Report Numeri

Metrica	Valore	Interpretazione
Accuracy	<i>0.97</i>	Il modello classifica correttamente il 97% delle immagini totali nel test set.
Macro Avg Precision	<i>0.98</i>	Media della precisione calcolata indipendentemente per ogni classe; indica un'altissima affidabilità nelle previsioni positive.
Macro Avg Recall	<i>0.97</i>	Capacità media del modello di individuare correttamente i numeri presenti (sensibilità media).
Macro Avg F1-Score	<i>0.97</i>	Media armonica tra precisione e recall, che conferma l'ottima robustezza del modello su tutte le classi.

Tabella 4: Metriche globali del test set - Numeri (14 Classi).

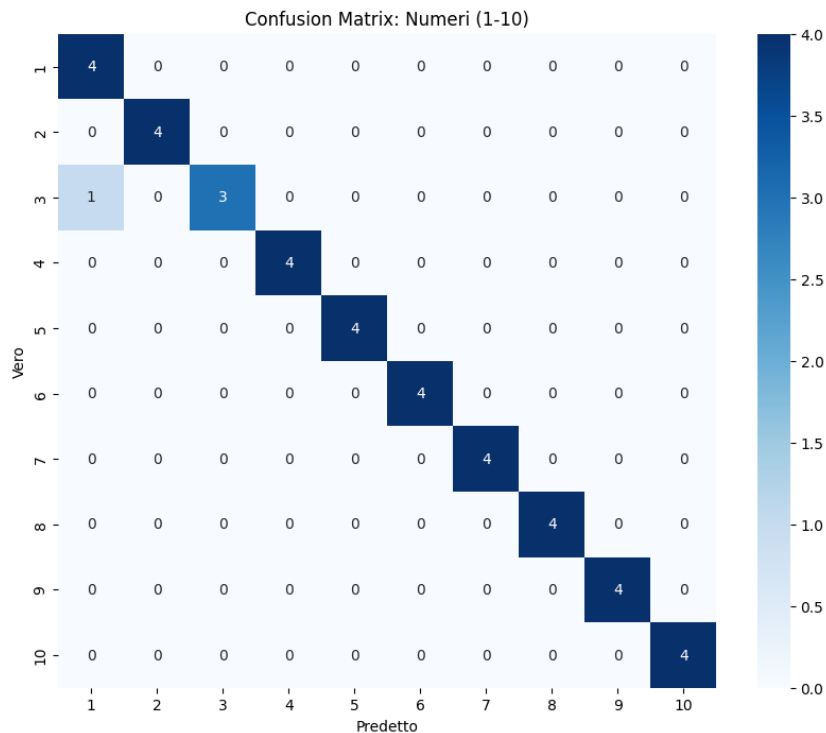


Figura 5: Matrice di Confusione - Numeri

4.2.2 Report Lettere

Metrica	Valore	Interpretazione
Accuracy	<i>1.00</i>	Il modello ha classificato correttamente il 100% delle immagini totali.
Macro Avg Precision	<i>1.00</i>	Precisione massima: non sono stati generati falsi positivi per nessuna delle classi.
Macro Avg Recall	<i>1.00</i>	Sensibilità massima: il modello ha individuato correttamente tutte le istanze presenti nel set.
Macro Avg F1-Score	<i>1.00</i>	Punteggio perfetto che indica l'assenza di errori tra precisione e recall.

Tabella 5: Metriche globali del test set - Lettere (14 Classi).



Figura 6: Matrice di Confusione - Lettere

4.2.3 Analisi sulle metriche

Le metriche ottenute dall'addestramento basato sul dataset a 14 classi (10 numeri + 4 semi) confermano le ipotesi formulate in fase di progettazione. La riduzione della complessità dello spazio delle etichette ha permesso alla rete di estrarre feature più distintive e meno correlate, beneficiando di un numero maggiore di esempi per singola classe.

Il passaggio da una classificazione a 40 classi (combinazione univoca valore-seme) a una struttura a 14 classi ha prodotto un significativo miglioramento nella capacità di generalizzazione del modello. L'accuratezza sul set di validazione ha raggiunto picchi superiori al 95%, superando ampiamente le prestazioni ottenute con il modello a 40 classi.

Dall'analisi della Loss, si osservano oscillazioni marcate durante l'addestramento; questo comportamento è riconducibile alla dimensione ridotta del dataset e un learning rate

troppo alto. Questi due fattori rendono le metriche estremamente sensibili alle caratteristiche di singoli campioni "difficili" all'interno dei batch. Nonostante tale varianza, il modello dimostra una stabilità di apprendimento complessivamente superiore: disaccoppiando il riconoscimento del numero da quello del seme, la rete apprende con maggiore efficacia le gerarchie visive, identificando correttamente l'identità della carta indipendentemente dal suo contesto cromatico o simbolico.

5 Sperimentazione finale

La prima sperimentazione evidenzia come il limite principale dell'intero processo sia riconducibile all'esiguità del dataset originale. La scarsità di campioni ha influenzato le prestazioni dei modelli in due modi differenti.

Nel modello a 40 classi, ogni singola classe disponeva di un numero insufficiente di esempi per permettere alla rete di generalizzare. Di conseguenza, il modello è andato in overfitting, imparando a memoria i dettagli specifici del train set.

Nel modello a 14 classi, la riduzione delle classi ha mitigato l'overfitting, permettendo al modello di osservare più esempi per la stessa categoria (ad esempio, tutte le varianti di un "Asso" indipendentemente dal seme). Tuttavia, la dimensione limitata del dataset e un learning rate alto continuano a manifestarsi attraverso un'estrema instabilità della funzione di loss. La presenza di pochi esempi provoca picchi improvvisi nell'errore, rendendo la curva di apprendimento meno lineare e più sensibile alla varianza statistica del batch.

Inoltre, si è osservata l'assenza di esempi sufficienti per la valutazione delle prestazioni. La classificazione di ciascuna carta è valutata solo 2 volte rendendo il task più semplice. Per questa motivazione, sono stati raccolti altri mazzi di carte con l'obiettivo di aumentare il dataset portando ad una suddivisione [50% 17% 33%].

- Train: 6 mazzi
- Valid: 2 mazzi
- Test: 4 mazzi

Dopo aver effettuato il processo di data augmentation di seguito è mostrata la nuova distribuzione:

- Train: 2640 immagini
- Valid: 480 immagini
- Test: 160 immagini

5.0.1 Configurazione degli Iperparametri e modifiche strutturali

Per l'addestramento del modello disaccoppiato (Numeri e Semi), sono stati adottati i seguenti parametri di configurazione:

5.0.2 Configurazione degli Iperparametri

Il processo di addestramento è stato configurato con una serie di iperparametri scelti per bilanciare la velocità di convergenza e la stabilità del modello.

Iperparametro	Descrizione e Motivazione
Batch Size	<i>32</i> – Definisce quante immagini vengono caricate in memoria contemporaneamente (approccio <i>mini-batch</i>). Ottimizza l'uso della GPU e stabilizza il gradiente.
Num Classes	<i>40</i> – Rappresenta il numero totale di categorie (10 valori per 4 semi).
Num Classes (Valore)	<i>10</i> – Classi relative al valore numerico della carta (dall'asso al re).
Num Classes (Seme)	<i>4</i> – Classi relative al seme della carta (A, B, C, D).
Epochs	<i>30</i> – Numero massimo di iterazioni complete sul dataset.
Learning Rate	<i>0.0001</i> – Regola l'entità degli aggiornamenti dei pesi.
Patience	<i>6</i> – Numero di epoche consecutive senza miglioramenti della <i>validation loss</i> prima di interrompere l'addestramento.
Min Delta	<i>0.0001</i> – La variazione minima della <i>loss</i> considerata come un miglioramento significativo.

Tabella 6: Sintesi degli iperparametri di addestramento.

Per migliorare ulteriormente l'addestramento, sono state introdotte due modifiche strutturali alla pipeline. La prima operazione è l'inserimento di uno scheduler dinamico del learning rate con l'utilizzo di ReduceLROnPlateau (con factor=0.1 e patience=2) che risponde alla necessità di gestire l'instabilità della loss riscontrata nei primi test.

Lo scheduler monitora la Validation Loss, se questa non migliora per 2 epoche consecutive, il learning rate viene ridotto di un fattore 10 (passando ad esempio da 10^{-4} a 10^{-5}). Questa tecnica permette al modello di rallentare la modifica dei pesi quando si avvicina a un minimo della funzione di errore permettendo un assestamento più preciso dei pesi e riducendo le oscillazioni violente viste nei grafici precedenti.

Nella prima sperimentazione, solo l'ultimo strato lineare (Fully Connected) veniva addestrato, mantenendo invariata tutta la struttura della ResNet. In questa sperimentazione è stato sbloccato il Layer 4 (l'ultimo blocco di convoluzioni della ResNet). I primi layer di una rete convoluzionale imparano concetti universali (bordi, colori, linee) gli ultimi layer (come il Layer 4), invece, imparano a riconoscere forme complesse e specifiche. Sbloccando il Layer 4, permettiamo alla rete di adattare i suoi filtri visivi di alto livello alle caratteristiche peculiari del dominio delle carte napoletane (come le decorazioni dei Re o la forma specifica dei Bastoni).

In questa fase finale di sperimentazione si è voluto focalizzare anche sul confronto prestazionale tra due architetture della stessa famiglia: la ResNet-18 e la ResNet-50. L'obiettivo di questo test comparativo non è solo identificare il modello con l'accuratezza più elevata, ma anche fornire soluzioni versatili a seconda del target hardware di destinazione.

5.1 Modello a 40 classi (ResNet18)

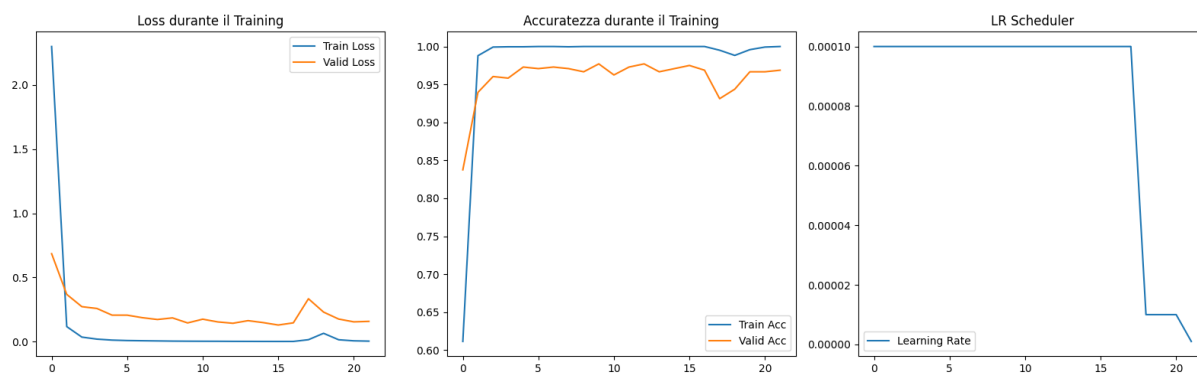


Figura 7: Funzione di Loss - Funzione di accuratezza - Learning Rate Scheduler

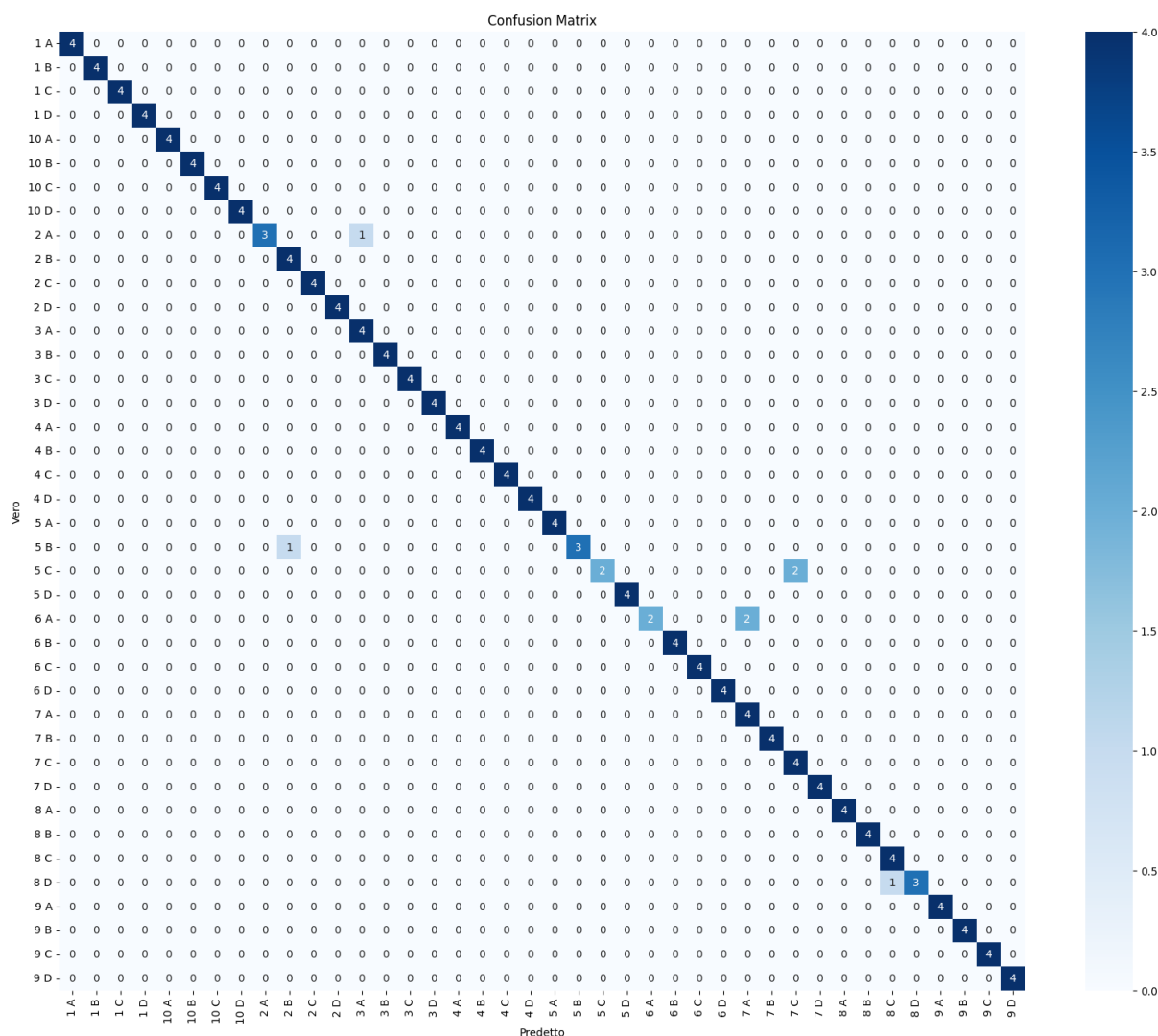


Figura 8: Matrice di confusione

Report Finale su Test Set La Tabella 7 mostra il salto prestazionale ottenuto con il modello ottimizzato, che raggiunge un'accuratezza quasi perfetta.

Metrica	Valore	Interpretazione
Accuracy	<i>0.96</i>	Il modello classifica correttamente il 96% delle immagini. Un incremento del 31% rispetto al modello base.
Macro Avg Precision	<i>0.97</i>	Indica che quando il modello predice una classe, è corretto nel 97% dei casi.
Macro Avg Recall	<i>0.96</i>	Capacità del modello di identificare correttamente quasi tutti gli esempi reali delle classi.
Macro Avg F1-Score	<i>0.95</i>	Sintesi armonizzata che conferma l'assenza di sbilanciamenti significativi tra le classi.

Tabella 7: Metriche globali del test set finale (Modello Ottimizzato).

5.2 Modello a 14 classi (ResNet18)

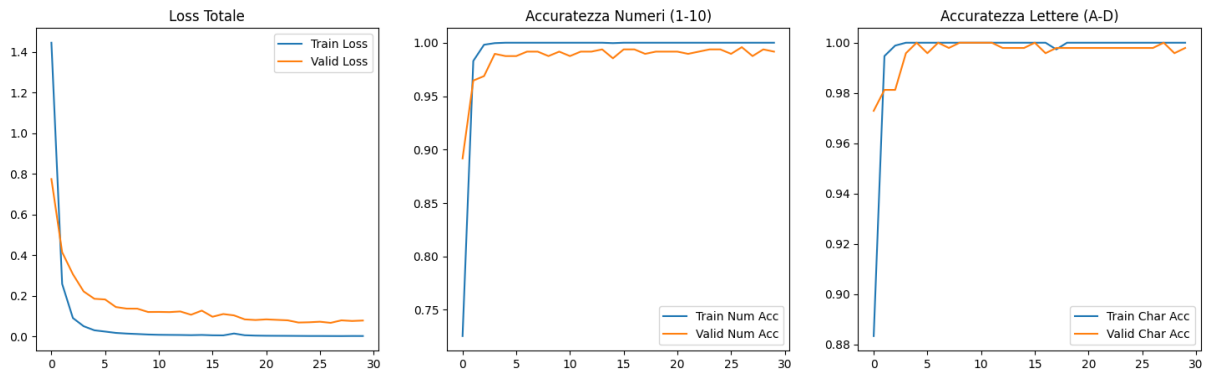


Figura 9: Funzione di Loss - Funzione di accuratezza - Learning Rate Scheduler

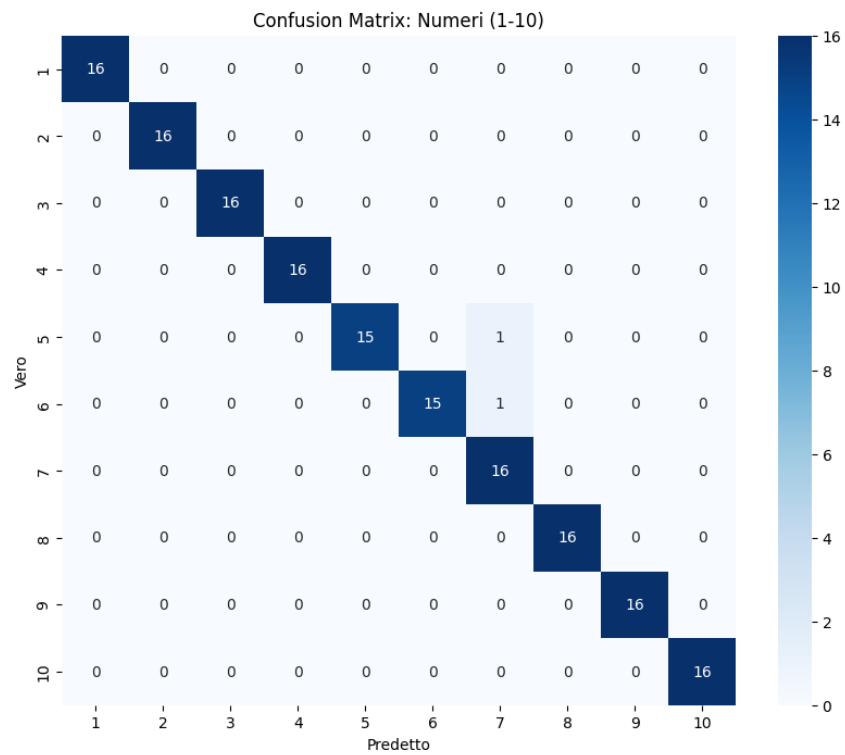


Figura 10: Matrice di confusione sui Numeri



Figura 11: Matrice di confusione sui semi

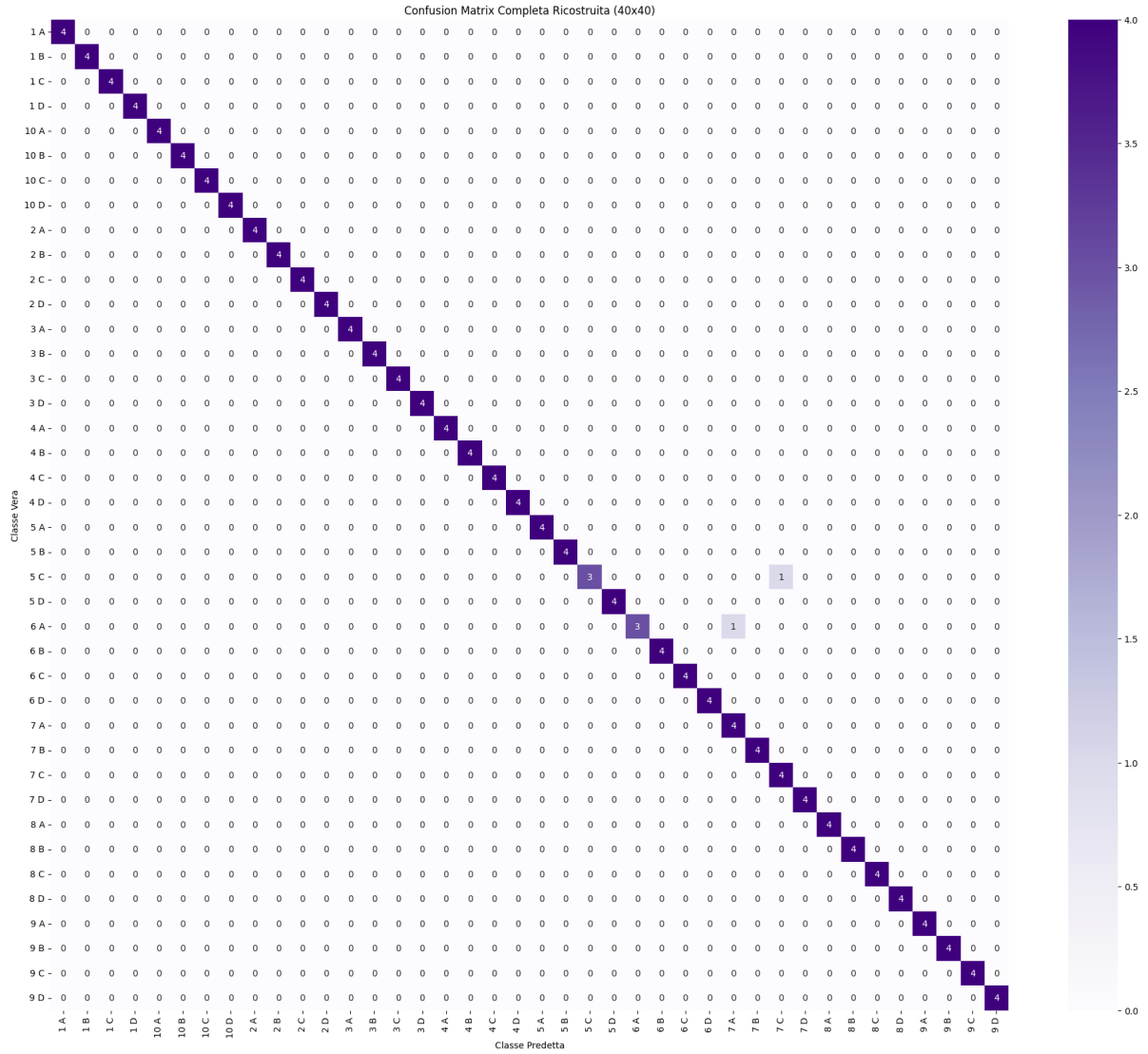


Figura 12: Matrice di Confusione Generale

Report Finale su Test Set (40 Classi) La Tabella 8 mostra i risultati definitivi ottenuti con la configurazione ottimizzata. Il modello raggiunge prestazioni quasi perfette, dimostrando una capacità di discriminazione eccellente su tutte le 40 categorie.

Metrica	Valore	Interpretazione
Accuracy	<i>0.99</i>	Il modello classifica correttamente il 99% dei campioni di test. Solo 2 immagini su 160 sono state errate.
Macro Avg Precision	<i>0.99</i>	Indica che le predizioni sono estremamente affidabili per tutte le classi, con quasi zero falsi positivi.
Macro Avg Recall	<i>0.99</i>	Il modello identifica correttamente la quasi totalità degli esempi reali per ogni combinazione seme-valore.
Macro Avg F1-Score	<i>0.99</i>	Certifica una stabilità pressoché perfetta del classificatore su tutto lo spettro delle 40 classi.

Tabella 8: Metriche globali del test set finale (Accuratezza 99%).

5.3 Modello a 40 classi (ResNet50)

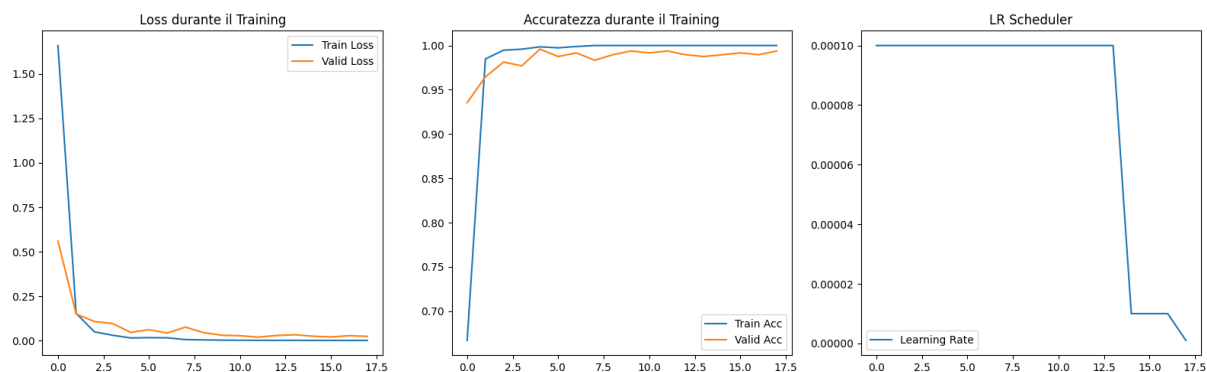


Figura 13: Funzione di Loss - Funzione di accuratezza - Learning Rate Scheduler

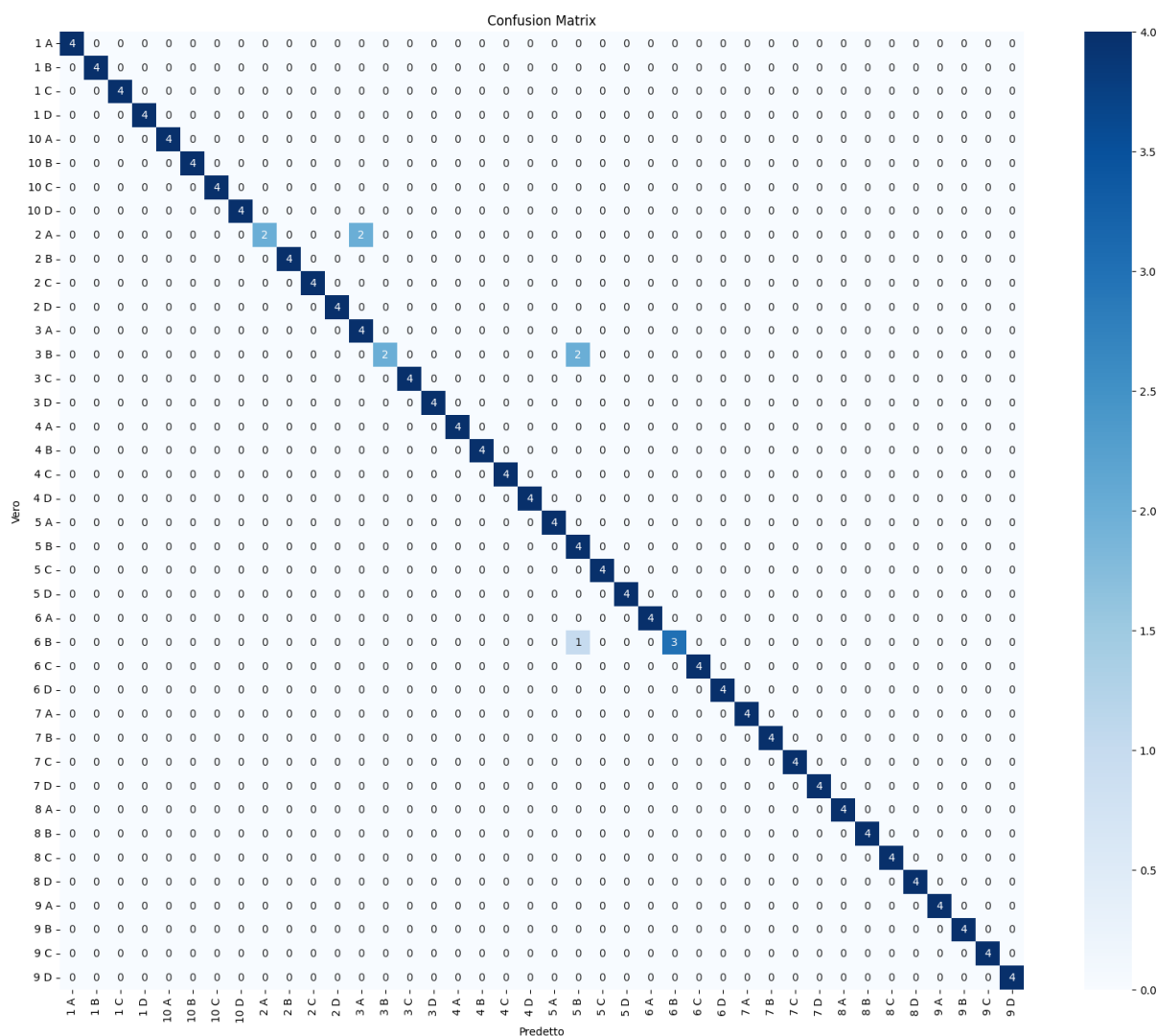


Figura 14: Matrice di Confusione

Report Finale su Test Set (Configurazione 40 Classi) La Tabella 9 sintetizza le prestazioni globali ottenute. Il modello dimostra una capacità di discriminazione quasi assoluta, con un'accuratezza del 97% su un totale di 160 campioni di test.

Metrica	Valore	Interpretazione
Accuracy	<i>0.97</i>	Il modello classifica correttamente il 97% dei test. La robustezza è confermata dalla dimensione del supporto (4 per classe).
Macro Avg Precision	<i>0.98</i>	L'alta precisione indica una bassissima incidenza di falsi positivi tra le 40 categorie.
Macro Avg Recall	<i>0.97</i>	Capacità quasi totale di recuperare correttamente gli esempi appartenenti a ogni classe.
Macro Avg F1-Score	<i>0.97</i>	Valore di sintesi che conferma l'eccellenza del modello nel bilanciare precisione e richiamo.

Tabella 9: Metriche globali del test set (Accuratezza 97%).

5.4 Modello a 14 classi (ResNet50)

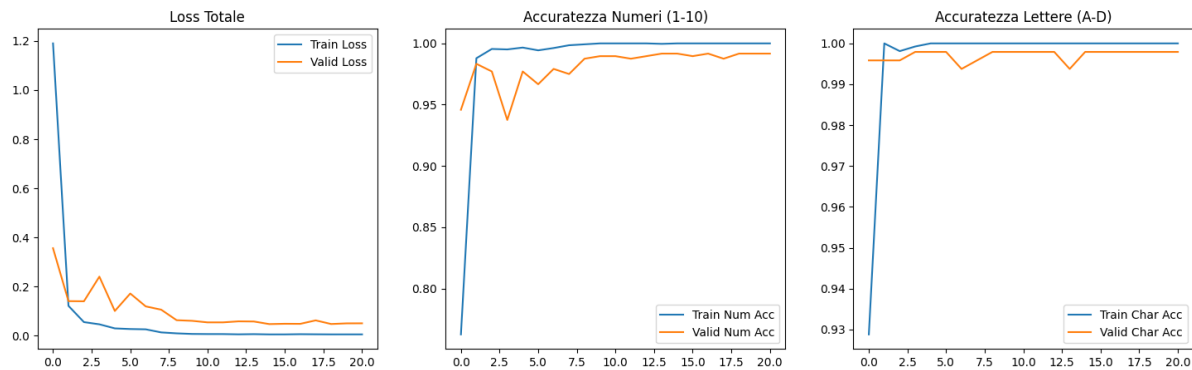


Figura 15: Funzione di Loss - Funzione di accuratezza - Learning Rate Scheduler

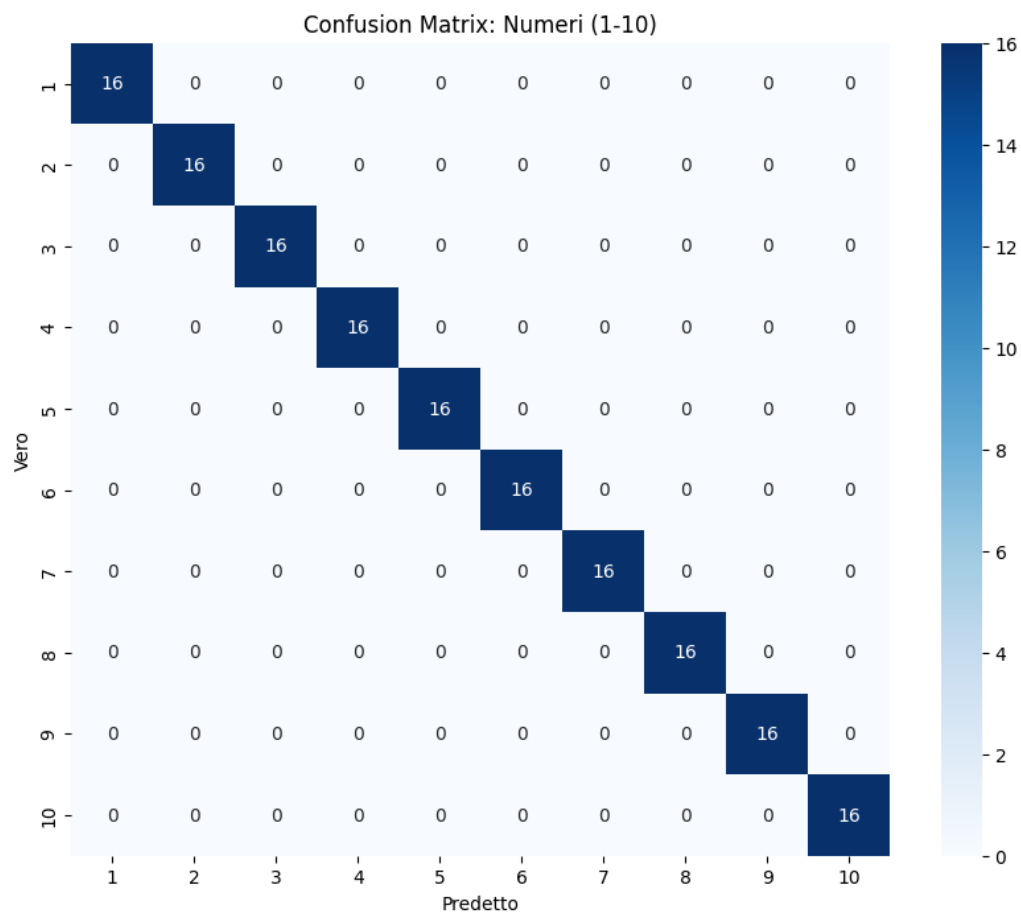


Figura 16: Matrice di Confusione sui Numeri

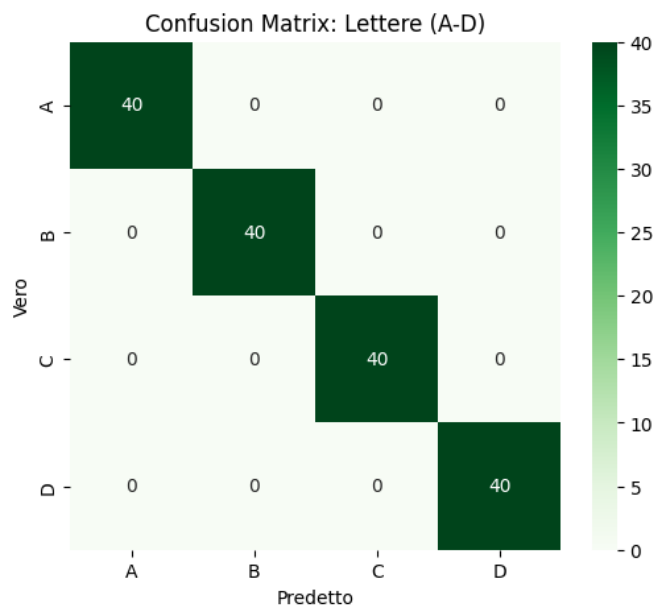


Figura 17: Matrice di Confusione sui Semi

Metrica	Valore	Interpretazione
Accuracy	<i>1.00</i>	Il modello ha classificato correttamente il 100% dei campioni di test (160 su 160).
Macro Avg Precision	<i>1.00</i>	Assenza totale di falsi positivi: ogni predizione effettuata dal modello è risultata esatta.
Macro Avg Recall	<i>1.00</i>	Assenza totale di falsi negativi: il modello ha individuato correttamente ogni singola carta presente.
Macro Avg F1-Score	<i>1.00</i>	Rappresenta la perfetta armonia tra precisione e richiamo su tutto lo spettro delle classi.

Tabella 10: Metriche globali del test set finale (Accuratezza 100%).

6 Explainable AI

In questa fase vogliamo visualizzare come il modello riconosce le carte e su quali punti si focalizza maggiormente per prendere decisioni.

Per ottenere questo risultato, abbiamo integrato la libreria Captum di PyTorch, che fornisce strumenti di Explainable AI.

Nel caso del modello *multi-head* (testa 'numeri' + testa 'lettere'), l'analisi viene eseguita separatamente sulle due uscite: per ogni immagine calcoliamo una mappa di attribuzione per la predizione del numero e una per la predizione della lettera.

Abbiamo scelto di usare due approcci complementari per ottenere una visione robusta dell'importanza delle feature.

- *Integrated Gradients (IG) + NoiseTunnel (SmoothGrad-Squared)* Metodo basato sui gradienti. IG assegna un valore di importanza ai pixel calcolando l'integrale del gradiente dell'output lungo il percorso che va dalla baseline x' all'input x . Per ridurre la variabilità e il rumore delle mappe, applichiamo *NoiseTunnel* a IG con `nt_type = smoothgrad_sq`: vengono aggiunte piccole perturbazioni gaussiane all'input, si calcolano più volte le attribuzioni e si aggregano (nel caso *sq* si enfatizzano regioni con contributi consistenti).

$$IG_i(x) = (x_i - x'_i) \times \int_0^1 \frac{\partial F(x' + \alpha(x - x'))}{\partial x_i} d\alpha$$

- x_i : È il valore della feature i -esima (ad esempio, l'intensità di un pixel) nell'input originale.
- x'_i : È il valore della stessa feature nella baseline. Nel nostro caso l'input è *normalizzato* (`Normalize(MEAN, STD)`), quindi una baseline a zero corrisponde a un'immagine "neutra" nello *spazio normalizzato* (nello spazio dei pixel equivalente, canale per canale, al valore medio usato nella normalizzazione).
- α : È il parametro di interpolazione che varia da 0 a 1, guidando il percorso dalla baseline all'input.

- $\frac{\partial F(x' + \alpha(x - x'))}{\partial x_i}$: Rappresenta il gradiente della funzione del modello rispetto a quella specifica feature, valutato nei vari punti intermedi lungo il percorso.

- *Occlusion* Approccio basato su perturbazioni locali. Fa scorrere una finestra sull'immagine (dimensione 40×40) con stride di 10 pixel, sostituendo la regione occlusa con una baseline costante (zeri per canale nello spazio normalizzato). L'attribuzione misura quanto cambia l'output del modello al variare dell'occlusione: regioni la cui rimozione riduce maggiormente la confidenza risultano più rilevanti per la classificazione.

L'explainability è stata applicata all'intero test set del dataset. Per ogni immagine generiamo e salviamo una figura riepilogativa (layout 2×3): nella prima riga le mappe per la testa 'numeri' (IG + NoiseTunnel / SmoothGrad-Squared e Occlusion) e nella seconda riga le mappe per la testa 'lettere', sempre affiancate all'immagine originale. I risultati vengono infine compressi in un archivio ZIP.

6.1 Analisi dei risultati

6.1.1 Guida come analizzare i risultati

Le mappe ottenute dalle tecniche XAI presentano due colori diversi. I colori non indicano un'intensità generica, ma il peso (positivo o negativo) che una certa area ha avuto sulla predizione finale:

- *Verde (Contributo Positivo)* Sono le zone che il modello considera fondamentali per confermare la sua scelta. Più il verde è intenso, più quei pixel hanno convinto che la carta appartenga a quella specifica classe.
- *Rosso (Contributo Negativo / Disturbo)* Sono le zone che creano confusione. Indicano elementi visivi che abbassano la confidence della predizione. Se un'area è rossa, significa che quei pixel hanno abbassato la confidenza del modello o gli hanno suggerito una classe sbagliata.

6.1.2 Risultati

Analizzando le mappe generate dalle immagini dei test, emergono molti punti favorevoli:

- *Indipendenza dallo sfondo* In molti casi l'attivazione principale risulta confinata alla regione della carta, con contributo limitato dello sfondo.
- *Analisi con IG + NoiseTunnel (SmoothGrad-Squared)* Le mappe IG (stabilizzate con NoiseTunnel) evidenziano in modo coerente le zone più informative per la predizione; tipicamente includono bordi, simboli e dettagli interni utili al riconoscimento.
- *Analisi con Occlusion* Evidenzia la distribuzione del peso semantico sulle diverse regioni della carta. Nell'esempio (Fig. 19), l'oscuramento della figura centrale determina un crollo della confidenza, validandola come area critica.

Per entrambe le tecniche, l'approccio multi-head rivela una specializzazione funzionale delle feature: mentre per il seme (Char) il modello focalizza l'attenzione sul bastone del cavaliere, per il numero (Num) la regione discriminante si sposta sulla sagoma del cavallo.

Questa divergenza dimostra che il modello ha appreso strategie di estrazione differenziate e coerenti con i due diversi compiti di classificazione.

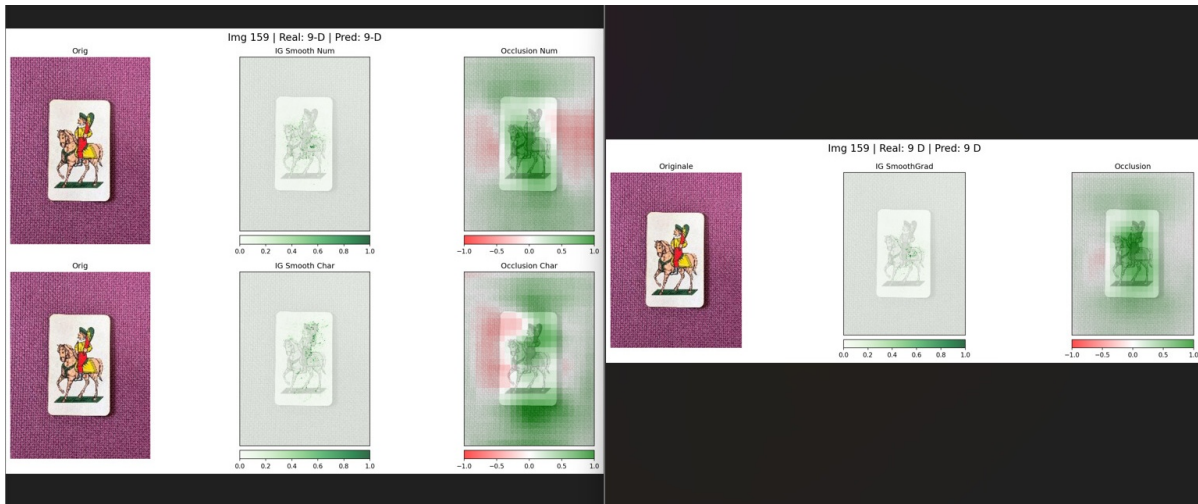


Figura 19: XAI 40 classi vs Multi-Head

6.2 Il modello generalizza?

Per verificare la robustezza e la capacità di generalizzazione del modello è stato svolto un test utilizzando il mazzo di Napoletane originali di Topolino. Nello specifico, si tratta di una reinterpretazione in chiave disneyana delle tradizionali carte napoletane, realizzata dall'artista partenopeo Blasco Pisapia. Queste illustrazioni introducono feature morfologiche e cromatiche molto diverse da quelle originali, rendendo l'interpretazione non banale anche per noi.

L'analisi XAI su queste carte (esempio in figura, predizione "1-A") fornisce informazioni importanti sul comportamento del modello in scenari totalmente di fuori della propria zona di comfort.

Il modello, nell'esempio mostrato in 20, mostra una differenza importante nelle confidenze delle due teste. Mentre la classificazione del seme ("A") mantiene una confidenza altissima (99.5%), la predizione del valore numerico (asso) crolla al 47.5%. Questo calo riflette oggettivamente la difficoltà interpretativa.

Le mappe confermano che il modello tenta comunque di estrarre feature di alto livello, concentrandosi sui bordi ad alto contrasto delle nuove figure (i contorni dei personaggi e l'oggetto circolare centrale), cercando pattern riconducibili alle classi note.

La mappa di Occlusion Num è particolarmente rivelatrice. Si nota una vasta area di attivazione negativa in corrispondenza della figura centrale. Nell'analisi di occlusione, un'area rossa indica che la presenza di quegli specifici pixel penalizza la predizione della classe target. In altre parole, la XAI ci mostra visivamente che il modello riconosce i personaggi disneyani come elementi estranei e porta fuori strada la classificazione dell'asso. Al contrario, l'ampia area verde nella Occlusion Char dimostra che il modello riesce ancora a trovare sufficienti feature globali per confermare il seme con sicurezza.

Questo test dimostra come il modello non si limiti a una memorizzazione rigida del dataset originale, ma possieda una buona capacità di generalizzazione, mentre le tecniche XAI si rivelano fondamentali per diagnosticare in modo interpretabile le cause dei cali di confidenza su dati anomali.



Figura 20: XAI su Asso di Denari dal mazzo di carte di Topolino

7 Conclusioni

Le diverse sperimentazioni condotte hanno permesso di avere un percorso evolutivo nel rendere l'idea fattibile ottenendo risultati accettabili nel riconoscimento delle carte napoletane. Il più grande ostacolo è stata la scarsità di dati poichè l'intero dataset è stato raccolto manualmente, ma con una buona combinazione di Data Augmentation, la scelta di buone combinazioni di iperparametri e l'uso della ResNet basata su ImageNet ha permesso di raggiungere l'obiettivo.

7.1 Approccio a 40 classi VS Approccio a 14 classi

- *Approccio a 40 Classi (Single-Task)* Rappresenta la soluzione più scontata e intuitiva, ma anche quella che necessita di più dati. Sebbene con le dovute ottimizzazioni (Fine-Tuning e Scheduler) abbia raggiunto una precisione finale elevata, ha mostrato inizialmente una forte propensione all'overfitting.
- *Approccio a 14 Classi (Multi-Label/Disaccoppiato)* Fin dalla prima sperimentazione appare la strategia più adeguata per la gestione di un dataset più limitato. Riducendo il numero di classi, il modello ha potuto sfruttare un numero maggiore di esempi per ogni singola classe (es. tutti gli "Assi" contribuiscono alla stessa classe indipendentemente dal seme). Questo ha garantito una convergenza più rapida e una capacità di generalizzazione superiore nelle prime fasi del progetto.

7.2 ResNet-18 vs ResNet-50

La scelta dell'architettura gioca un ruolo chiave nel bilanciamento tra precisione e usabilità:

- *ResNet-18* Nonostante la profondità limitata, è riuscita a saturare le prestazioni. Rappresenta la scelta ottimale per il deployment su dispositivi mobili o sistemi embedded, dove la velocità di inferenza e il risparmio di memoria sono importanti.
- *ResNet-50* Questa rete ha mostrato una stabilità superiore nella gestione dei dettagli più complessi delle figure. Sebbene richieda maggiori risorse computazionali, è il modello di riferimento per scenari in cui è necessaria la massima affidabilità statistica, tipicamente in ambienti server o desktop. Questa rete in combinazione con l'approccio a 14 classi ha ottenuto i migliori risultati possibili.